	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 07

No de Prácticas(s): 11

Integrante(s): 323078401

323115296

323082392

323139441

323253916

No. de lista o brigada: 04

Semestre: 2027-I

Fecha de entrega: 13/11/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **3**

1.1. Planteamiento del problema 3

1.2. Motivación 3

1.3. Objetivos 3

2. Marco Teórico **4**

3. Desarrollo **5**

3.1. Actividad 1: Análisis de las respuestas de las LLM 5

3.2. Actividad 2 6

4. Resultados **8**

5. Conclusiones **9**

6. Referencias **9**

1. Introducción

1.1 Planteamiento del problema

El problema abordado en esta práctica comprende dos actividades complementarias. Por un lado, se requiere analizar las explicaciones y propuestas de código generadas por distintos modelos de lenguaje al consultarles cómo implementar cuadros de entrada de texto en interfaces gráficas de usuario (GUI) mediante las bibliotecas Swing y JavaFX. Por otro lado, se busca conseguir la lectura de un archivo de texto plano con extensión **.txt**, contabilizar el total de palabras contenidas en él y desplegar tanto el texto íntegro como el resultado numérico del conteo dentro de una interfaz gráfica.

1.2 Motivación

Manejar archivos externos y su integración con interfaces visuales para desarrollar programas capaces de persistir información e interactuar con el usuario más allá de la terminal de comandos. Comprender cómo crear flujos de lectura de caracteres con mecanismos de control de excepciones para evitar interrupciones inesperadas ante archivos inexistentes o inaccesibles, y mantener el principio de encapsulamiento con la separación entre la lógica de procesamiento y la capa gráfica. Asimismo, comparar respuestas de distintas herramientas de inteligencia artificial para identificar la utilidad de las respuestas obtenidas.

1.3 Objetivos

- Analizar y comparar las respuestas generadas por distintos modelos de lenguaje al solicitarles cómo crear espacios de cuadro de texto en Java utilizando las bibliotecas Swing y JavaFX.
- Implementar una clase encargada de abrir, validar y leer un archivo con extensión **.txt** ; y contabilizar el número total de palabras contenidas en el archivo analizado.
- Construir una interfaz gráfica de usuario en Java Swing que permita visualizar tanto el contenido del archivo como el total de palabras contabilizadas.

2. Marco Teórico

La construcción de interfaces visuales se fundamenta en la programación dirigida por eventos, un paradigma en el cual el flujo de ejecución responde a acciones generadas por el usuario sobre los controles gráficos, como presionar un botón [1]. La biblioteca Swing, integrada de forma nativa en el entorno estándar de Java, proporciona contenedores como **JFrame** y paneles organizadores como **JPanel**, los cuales emplean administradores de diseño (*Layout Managers*) como **BorderLayout** o **FlowLayout** para distribuir espacialmente los componentes en pantalla [1].

Para la captura y despliegue de información textual dentro de una interfaz gráfica, Swing dispone de controles especializados derivados de la jerarquía de componentes de texto [2]. Mientras que **JTextField** está diseñado para la edición de una sola línea de texto, **JTextArea** permite almacenar y visualizar múltiples líneas de contenido, pudiendo configurarse en modo de solo lectura y alojarse dentro de un contenedor **JScrollPane** para habilitar barras de desplazamiento cuando el texto supera las dimensiones visibles de la ventana [2]. La comunicación entre la interacción del usuario y la lógica de la aplicación se aplica mediante interfaces escuchadoras de eventos (*listeners*), como **ActionListener**, cuyo método **actionPerformed** se invoca automáticamente cada vez que el componente asociado dispara una acción [3].

Por otro lado, el acceso a información almacenada en medios secundarios requiere el uso de flujos de entrada y salida (*I/O Streams*), los cuales abstraen una fuente de datos como una secuencia continua de información [4]. Cuando se trabaja con archivos de texto plano, Java ofrece flujos orientados a caracteres a través de clases como **FileReader**, que efectúa la lectura directa desde un objeto **File** interpretando la codificación de caracteres del sistema [5]. Dado que realizar operaciones individuales directamente sobre el disco implica una alta sobrecarga, resulta habitual envolver el flujo base dentro de un **BufferedReader**, el cual incorpora un búfer en memoria intermedia y habilita la lectura eficiente de renglones completos mediante el método **readLine()** [6].

Finalmente, las operaciones de acceso a archivos son susceptibles a fallos externos, como rutas inexistentes o restricciones de lectura, los cuales son modelados en Java mediante excepciones comprobadas (*checked exceptions*) como **IOException** [7]. El compilador exige que este tipo de anomalías sean gestionadas explícitamente mediante bloques **try-catch** o declaradas en la firma del método, permitiendo que el programa intercepte el fallo, libere o proteja sus recursos y evite una terminación abrupta durante la ejecución [7].

3. Desarrollo

3.1 Actividad 1: Análisis de las respuestas de las LLM

La primera actividad consistió en consultar a tres modelos de lenguaje distintos sobre la manera de implementar cuadros de texto en interfaces gráficas de Java utilizando las bibliotecas Swing y JavaFX.

La primera consulta, realizada con el modelo Gemini Flash, se planteó mediante una instrucción directa y breve en la que se preguntó cómo crear un espacio de cuadro de texto en GUI para Java con Swing y JavaFX, sin especificar restricciones de extensión, nivel de detalle ni tipo de control.

Ante esta amplitud, el modelo respondió con un enfoque eminentemente práctico orientado a proporcionar código funcional de referencia rápida. La solución distinguió entre campos de una sola línea (**JTextField** y **TextField**) y áreas de múltiples líneas (**JTextArea** y **TextArea**), entregando dos programas completos ejecutables que incluían un botón con manejo de eventos para leer el contenido e imprimirlo en consola, además de enumerar métodos clave como **getText()**, **setText()** y **setPromptText()**. Si bien reprodujo correctamente el comportamiento esperado, omitió explicaciones detalladas sobre el funcionamiento interno de cada línea o la jerarquía de contenedores empleada.

La segunda consulta, efectuada con ChatGPT, solicitó de manera explícita una descripción paso a paso de cómo crear un cuadro de texto en Java con Swing y JavaFX, exigiendo explicar el funcionamiento de cada línea de código y de cada clase utilizada.

Siguiendo estas restricciones, el modelo estructuró un tutorial extenso, acompañando los ejemplos de una sola línea con tablas descriptivas de cada clase importada y un desglose secuencial de cada instrucción. Asimismo, aclaró detalles relevantes de implementación, como el hecho de que el parámetro numérico en el constructor de **JTextField** define una anchura sugerida en columnas y no un límite estricto de caracteres, e incorporó un apartado adicional para cuadros multilínea y una tabla comparativa entre Swing y JavaFX. No obstante, al centrar su extensión en la disección sintáctica de cada línea, los programas principales omitieron botones y manejadores de eventos activos para capturar el texto en tiempo de ejecución.

La tercera consulta, ejecutada con el modelo Gemini Pro, estableció una estructura organizada en tres etapas: explicar primero los conceptos indispensables para lograrlo, detallar después aspectos convenientes o buenas prácticas y presentar al final un ejemplo sencillo.

El modelo respetó estrictamente este ordenamiento, sintetizando inicialmente la necesidad de instanciar componentes dentro de contenedores gobernados por administradores de diseño y el uso de **getText()** y **setText()**. Posteriormente, abordó como buenas prácticas el acoplamiento con escuchadores de eventos (**ActionListener** y **setOnAction**) y el empleo de **JPasswordField** o **PasswordField** para ocultar información sensible, cerrando con dos ejemplos mínimos funcionales que integraron un campo de una línea y un botón de captura. Aunque este enfoque equilibró la fundamentación conceptual con ejemplos ejecutables, dejó fuera los componentes multilínea para mantener sencillez.

Al contrastar las tres interacciones, observamos que la ausencia de restricciones en el primer prompt favoreció una respuesta sintética centrada en el código, mientras que la exigencia de desglose línea por línea en el segundo produjo mayor detalle sintáctico a costa de omitir la interacción por eventos. Por su parte, la estructura de la tercera consulta permitió abordar los principios orientados a objetos con implementaciones breves y funcionales.

3.2 Actividad 2

Para cumplir con el requerimiento de la práctica, se separó mediante dos clases con responsabilidades diferentes: **AnalizadorDeArchivo**, encargada de la lógica de acceso y procesamiento del documento, y **Main**, responsable de construir y gestionar la interfaz visual en Swing.

La clase **AnalizadorDeArchivo** encapsula cuatro atributos privados que mantienen el estado del análisis: **archivo**, de tipo **File**, que representa la referencia física al documento en disco; **contenidoDelTexto**, de tipo **String**, que acumula el texto íntegro leído; **totalPalabras**, de tipo **int**, que lleva el registro numérico de las palabras detectadas; y **archivoListo**, una bandera booleana que indica si el archivo fue localizado y validado correctamente antes de intentar su lectura.

El constructor **AnalizadorDeArchivo(String rutaAbsoluta)** inicializa **contenidoDelTexto** como una cadena vacía y **totalPalabras** en cero. Dentro de un bloque **try-catch**, intenta asociar **archivo** utilizando la ruta recibida; en caso de que no exista en el directorio de trabajo actual, verifica mediante **getResource** si el archivo se encuentra dentro del paquete compilado para resolver su ruta mediante una **URI**. Si tras esta comprobación el archivo existe físicamente, asigna el valor **true** a **archivoListo**; de lo contrario, lanza de manera intencional una **IOException** que es capturada para establecer **archivoListo** en **false** y reportar el mensaje de error en la terminal.

El procesamiento del contenido se realiza en el método público **procesarArchi-**

vo(). Como medida preventiva, el método evalúa primero si **archivoListo** es **false**, en cuyo caso emite un aviso en consola y finaliza su ejecución mediante **return** sin intentar abrir flujos inválidos. Cuando el archivo está disponible, instancia un **FileReader** sobre el atributo **archivo** y lo envuelve en un **BufferedReader** dentro de un bloque **try-catch** para gestionar posibles fallos durante la lectura.

A través de un ciclo **while**, el método invoca repetidamente **lector.readLine()** asignando cada renglón a la variable local **linea** mientras el resultado sea distinto de **null**. En cada iteración realiza dos operaciones: primero concatena **linea** seguida de un carácter de salto de línea (`"\n"`) al atributo **contenidoDelTexto** para reconstruir la estructura original del documento; segundo, fragmenta **linea** mediante `linea.split(" ")` generando un arreglo de subcadenas llamado **partes**. Inmediatamente, un ciclo **for** recorre cada elemento de **partes** mediante un índice entero y verifica con una estructura **if** que su longitud sea mayor a cero. Esta validación descarta segmentos vacíos provocados por espacios consecutivos o renglones en blanco, incrementando **totalPalabras** en una unidad por cada palabra válida encontrada. Al alcanzar el final del archivo y salir del ciclo **while**, el método cierra el flujo invocando **lector.close()** y expone los resultados mediante los métodos consultores **getContenidoDelTexto()** y **getTotalPalabras()**.

Por su parte, la clase **Main** contiene el método de entrada **main**, donde se instancia un **JFrame** titulado **"Contador de Palabras"** con dimensiones de **500** por **400** píxeles, cierre configurado mediante **EXIT_ON_CLOSE** y un administrador de diseño **BorderLayout**. En la región superior (**BorderLayout.NORTH**) se coloca un **JPanel** que agrupa un botón **JButton** con el texto **"Leer prueba.txt"** y una etiqueta **JLabel** inicializada en **"Total de palabras: 0"**. En la región central (**BorderLayout.CENTER**) se integra un **JTextArea** configurado como no editable (**setEditable(false)**) dentro de un **JScrollPane** para permitir el desplazamiento vertical u horizontal del texto.

La vinculación entre la interfaz y la lógica de negocio se implementa asociando un **ActionListener** anónimo al botón de carga. Cuando el usuario presiona el botón, el método **actionPerformed** define la cadena **"prueba.txt"** como ruta fija, crea una nueva instancia de **AnalizadorDeArchivo** con dicha ruta y ejecuta **procesarArchivo()**. Una vez concluida la lectura, actualiza dinámicamente el texto de la etiqueta superior concatenando **"Total de palabras: "** con el valor entero devuelto por **getTotalPalabras()**, y asigna al área de texto central la cadena retornada por **getContenidoDelTexto()**.

Para comprobar el funcionamiento del sistema, se utilizó como entrada un archivo de texto denominado **prueba.txt** que contenía únicamente la frase **"Esto es una prueba de la practica 11."**. Al ejecutar la aplicación y accionar el botón de lectura, el

analizador dividió la línea por el carácter espacio en fragmentos no vacíos , actualizando la interfaz gráfica para mostrar el texto **“Esto es una prueba de la practica 11.”** en el cuadro central y **“Total de palabras: 8”** en la etiqueta superior.

4. Resultados

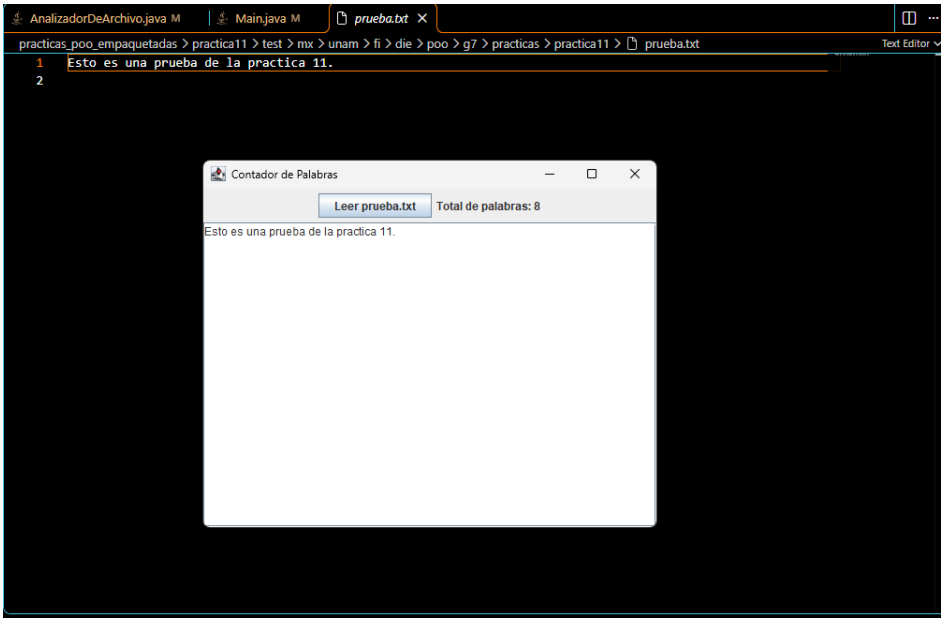


Figura 1: Ejecución de la interfaz gráfica mostrando el contenido leído del archivo **prueba.txt**.

Figura 2: Diagrama de clases UML

Ver imagen 01_diagrama_clases_uml.png en GitHub

Figura 3: Diagrama de clases UML con cardinalidad

Ver imagen 01_diagrama_clases_uml_cardinalidad.png en GitHub

Figura 4: Diagrama de flujo del constructor de AnalizadorDeArchivo

Ver imagen 02_flujo_analizador_constructor.png en GitHub

Figura 5: Diagrama de flujo de procesarArchivo de en AnalizadorDeArchivo

Ver imagen 03_flujo_analizador_procesarArchivo.png en GitHub

Figura 6: Diagrama de secuencia

Ver imagen 05_diagrama_secuencia.png en GitHub

Figura 7: Diagrama de flujo del Main actionPerformed

Ver imagen 04_flujo_main_actionPerformed.png en GitHub

5. Conclusiones

Los resultados obtenidos evidencian que la separación de responsabilidades entre la clase encargada del procesamiento de archivos y la interfaz gráfica permite mantener desacoplada la lógica de lectura respecto a la representación visual. El uso combinado de **FileReader** y **BufferedReader** facilitó la extracción secuencial del contenido línea por línea, mientras que la validación previa mediante bandera de estado y bloques **try-catch** protegió la ejecución frente a posibles excepciones de entrada y salida al localizar o leer el archivo.

Asimismo, la integración de componentes de Swing aplicados mediante un **ActionListener** demostró cómo la programación dirigida por eventos permite actualizar dinámicamente controles gráficos como **JLabel** y **JTextArea** a partir de atributos encapsulados en objetos de dominio. Por último, el contraste entre las respuestas de los distintos modelos de lenguaje confirmó que incorporar restricciones estructurales y niveles explícitos de detalle en los prompts produce explicaciones más equilibradas entre fundamentación teórica e implementación práctica.

6. Referencias

[1] Oracle, "Creating a GUI With Swing," *The Java™ Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/uiswing/

[2] Oracle, "Using Text Components," *The Java™ Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/uiswing/components/text.html

[3] Oracle, "Writing Event Listeners," *The Java™ Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/uiswing/events/index.html

[4] Oracle, "I/O Streams," *The Java™ Tutorials*, Oracle Corporation. [En línea].

Disponible en: https://docs.oracle.com/javase/tutorial/essential/io/

[5] Oracle, "Character Streams," *The Java™ Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/essential/io/charstreams.html

[6] Oracle, "Buffered Streams," *The Java™ Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/essential/io/buffers.html

[7] Oracle, "What Is an Exception?," *The Java™ Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/essential/exceptions/definition.html